

Intro to Process Scheduling

Process scheduling is fundamental to multi-programming operating systems, enhancing productivity by efficiently managing CPU allocation among various processes.

CPU Scheduler

Selects processes from the ready queue for execution when the CPU is idle.

Multi-programming

Keeps many processes in memory, switching the CPU to maximize utilization.

Made with GAMMA

Scheduling Approaches

Non-Preemptive

Dedicated CPU

Process holds CPU until termination or wait-state.

Starvation Risk

Long burst times can starve shorter processes.

Low Utilization

Inefficient CPU usage.

Preemptive

Time-Bound

CPU taken after time quantum, termination, or wait-state.

Less Starvation

Fairer distribution of CPU time.

High Utilization

Maximizes CPU productivity.

Made with GAMMA

Goals of CPU Scheduling

1 **Maximum CPU Utilization**

Keep the processor busy as much as possible.

2 **Minimum Turnaround Time**

Reduce the total time from process arrival to termination.

3 **Minimum Wait Time**

Minimize the time processes spend waiting for CPU access.

4 **Minimum Response Time**

Ensure quick initial response to user requests.

5 **Maximum System Throughput**

Complete more processes per unit of time.

Made with GAMMA

Key Scheduling Metrics



Throughput

Number of processes completed per unit time.



Arrival Time (AT)

Time a process enters the ready queue.



Burst Time (BT)

CPU time required for process execution.



Turnaround Time (TAT)

Total time from arrival to termination (CT - AT).



Wait Time (WT)

Time process waits for CPU (TAT - BT).



Response Time

Time from ready queue to first CPU allocation.



Completion Time (CT)

Time when a process terminates.

Made with GAMMA

First Come, First Serve (FCFS)

FCFS is a non-preemptive scheduling algorithm where the process that arrives first in the ready queue is executed first.

Simple Execution

Processes are served in the order they arrive.

Convoy Effect

A long burst time process can significantly increase the average wait time for subsequent shorter processes, leading to poor resource management.



Shortest Job First (SJF)

SJF prioritizes processes with the least burst time, aiming to reduce average waiting time.

Non-Preemptive SJF

- **Least Burst Time**

  Processes with the shortest estimated burst time are executed first.

- **Estimation Challenge**

  Accurate burst time estimation is difficult.

- **Convoy Effect Risk**

  Can still suffer if the initial process has a large burst time.

- **Starvation Possible**

  Longer jobs may never get executed.

Preemptive SJF

- **Reduced Starvation**

  More equitable distribution of CPU time.

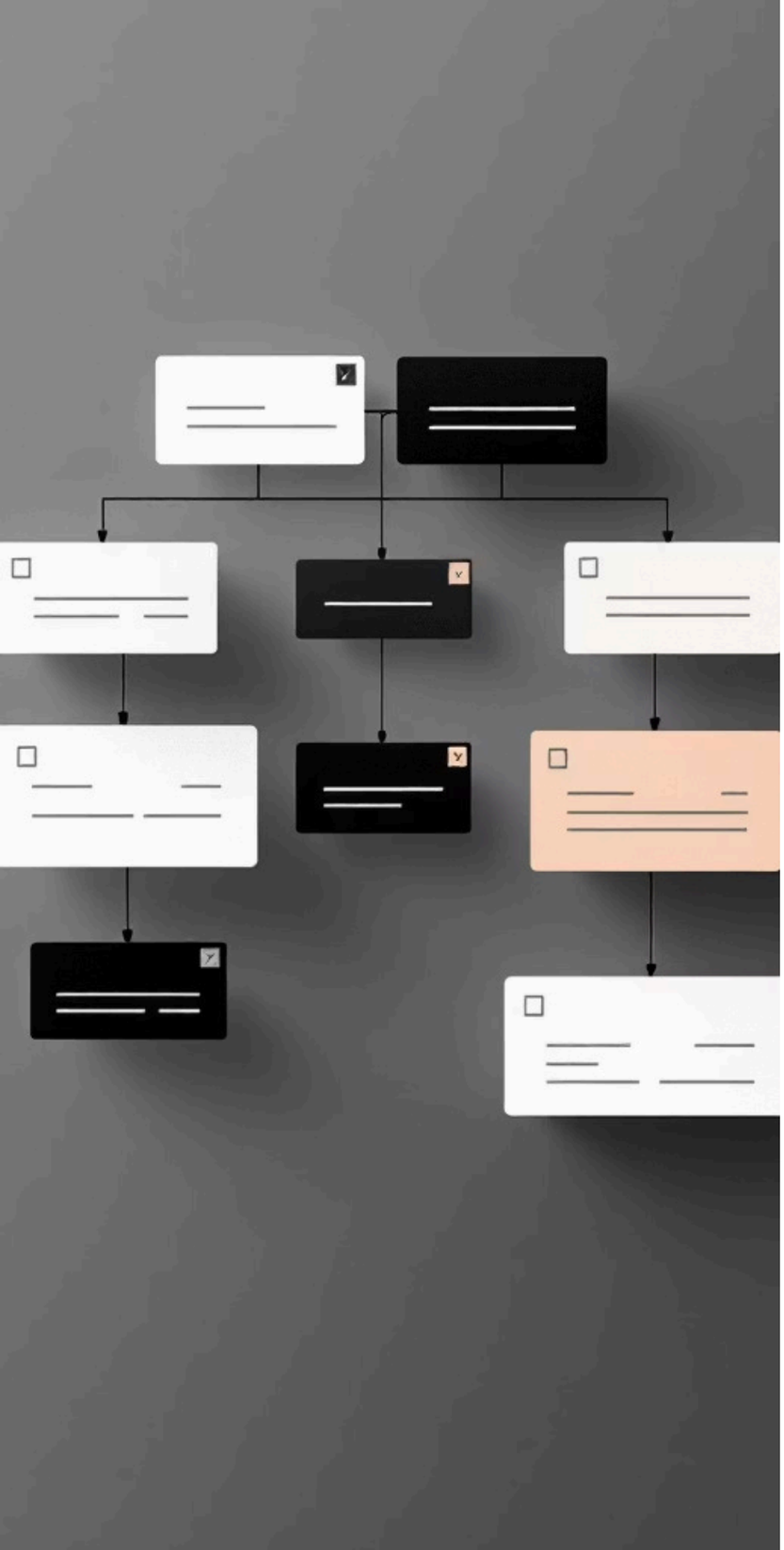
- **No Convoy Effect**

  Prevents long jobs from blocking short ones.

- **Lower Average WT**

  Scheduling short jobs first significantly decreases overall wait time.

Made with GAMMA



Priority Scheduling

Processes are assigned a priority, and the CPU is allocated based on this priority.

Non-Preemptive

- **Fixed Priority**

  CPU allocated to highest priority process until it releases the CPU.

- **SJF as Special Case**

  SJF is a form of priority scheduling where priority is inversely proportional to burst time.

Preemptive

- **Higher Priority Preemption**

  Current job is preempted if a new job with higher priority arrives.

- **Starvation Risk**

  Lower priority jobs may face indefinite waiting.

- **Ageing Solution**

  Gradually increasing the priority of long-waiting processes to prevent starvation.

Made with GAMMA

Round Robin Scheduling (RR)

A popular preemptive scheduling algorithm designed for time-sharing systems, ensuring fairness and low starvation.

1

FCFS with Preemption

Similar to FCFS but with a time quantum (TQ).

2

Time-Sharing Focus

Ideal for systems requiring quick response times.

3

Low Starvation

No process waits indefinitely; no convoy effect.

4

Easy Implementation

Straightforward to integrate into OS.

5

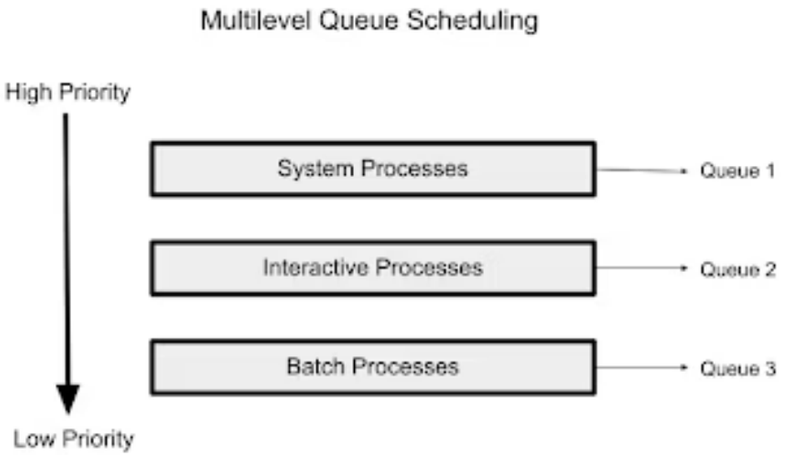
Context Switch Overhead

Small TQ leads to more context switches, increasing overhead.

Made with GAMMA

Multi-Level Queue (MLQ)

MLQ divides the ready queue into multiple sub-queues based on process properties like priority or type. Each queue has its own scheduling algorithm.



- **Fixed Assignment**

  Processes are permanently assigned to a queue.

- **Fixed Priority Preemptive**

  Scheduling among queues is fixed-priority preemptive (e.g., foreground has absolute priority over background).

- **Queue-Specific Algorithms**

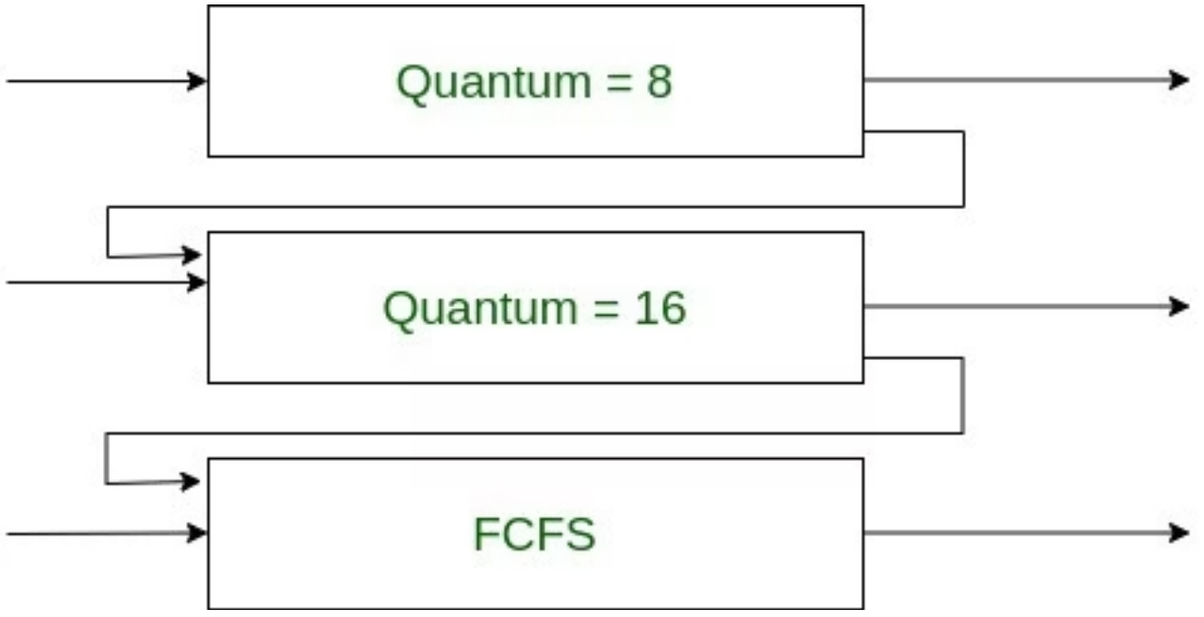
  Example: System processes (SP) use RR, Interactive processes (IP) use RR, Batch processes (BP) use FCFS.

- **Starvation & Convoy Effect**

  Lower priority processes can starve, and the convoy effect is present.

Made with GAMMA

Multi-Level Feedback Queue (MLFQ)



MLFQ allows processes to move between queues based on their CPU usage, separating processes by burst time characteristics. This prevents starvation through ageing.

- **Dynamic Queue Movement**

  Processes move to lower priority queues if they use too much CPU time.

- **I/O Bound Prioritization**

  I/O-bound and interactive processes remain in higher-priority queues.

- **Ageing Mechanism**

  Processes waiting too long in lower priority queues are moved up to prevent starvation.

- **Flexible & Configurable**

  Can be tailored to specific system requirements.

Made with GAMMA